Real-Time Network Intrusion Detection and Monitoring System

Snort 3 + Scapy + SQLite + Streamlit

Updated Project Documentation - Dashboard v3.0

Field	Value
Project	Real-Time Network Intrusion Detection System Dashboard
Latest dashboard version	dashboard-v3.0
Author	Maheen Nadeem
Main technologies	Snort 3, Scapy, SQLite, Streamlit, Python, Plotly
Latest update focus	Usability, page separation, JSON rule lookup, GitHub documentation and versioning

1. Project Description

The Real-Time Network Intrusion Detection and Monitoring System (NIDS) monitors live traffic in a VirtualBox lab, detects suspicious activity using Snort 3 Community Rules, and visualizes packet and alert data through an interactive Streamlit dashboard. The system combines live packet capture, signature-based detection, SQLite storage, automatic refresh, and rule-based alert explanation.

The project now also includes a preprocessed Snort rule documentation dataset. This dataset was created from the local Snort 3 Community Rules file and enriched with Snort.org documentation. It supports rule lookup, rule research, and future LLM/RAG-based alert explanation work.

The latest dashboard version, dashboard-v3.0, improves the dashboard interface by separating normal packet visibility from alert analysis and by allowing users to select alert rows directly to view documentation from the preprocessed JSON rule dataset.

2. Objectives

- Capture live traffic from the VirtualBox host-only lab network.
- Detect suspicious activity using Snort 3 Community Rules.
- Store packet and alert metadata in SQLite using rolling-window cleanup.
- Display live traffic trends, services, alert signatures, priorities, and detailed alert features.
- Separate normal traffic analysis from alert investigation in the dashboard interface.
- Make alert fields, signatures, CVEs, and rule documentation easier to understand.
- Preprocess and normalize Snort rule documentation, CVEs, MITRE mappings, references, and content patterns.
- Export rule documentation data in raw, preprocessed, normalized, and JSON formats.
- Use GitHub tags to maintain dashboard version history.

3. System Components

Component	Role	Purpose
Kali Linux VM - 192.168.56.10	Traffic generator	Generates normal and suspicious traffic such as ping, HTTP requests, Nmap scans, SSH probes, and service scans.
Ubuntu Victim VM - 192.168.56.20	Target machine	Receives traffic and hosts test services such as Apache/HTTP and SSH.
Ubuntu Snort/Dashboard VM - 192.168.56.30	Monitoring machine	Runs Snort 3, Scapy collector, alert collector, SQLite database, rule dataset, and Streamlit dashboard.
Windows Host	Access gateway	Uses VirtualBox NAT forwarding and Windows portproxy to

Component	Role	Purpose
		expose the Ubuntu VM dashboard to other reachable systems.

4. Current Architecture and Workflow

1. Kali sends traffic to the Ubuntu Victim VM over the host-only lab network.
2. The Ubuntu Snort/Dashboard VM observes traffic on interface enp0s8.
3. The Scapy packet collector captures packet metadata and stores it in the packets table.
4. Snort analyzes traffic against the local Community Rules file and produces JSON alerts when a rule/event matches.
5. The alert collector reads alert_json.txt, parses JSON lines, and inserts alert fields into SQLite.
6. The Streamlit dashboard reads SQLite and refreshes automatically to display packet visibility, alert analytics, selected-alert explanation, and raw JSON alerts.
7. For selected alerts, the dashboard retrieves rule documentation from the preprocessed JSON dataset using the selected rule SID.

```
Kali VM -> Ubuntu Victim VM
            \      /
             \    /
        Ubuntu Snort/Dashboard VM
        Snort 3 + Scapy + SQLite + Streamlit
        Rule documentation JSON lookup
```

5. Remote Dashboard Access

The dashboard is hosted inside the Ubuntu Snort/Dashboard VM. Windows does not host the Streamlit app directly. Instead, Windows acts as a forwarding gateway so other reachable systems can open the dashboard using the Windows host IP address.

```
Client Device / Mentor System
        -> Windows Host IP:8501
        -> Windows portproxy
        -> 127.0.0.1:8501
        -> VirtualBox NAT Port Forwarding
        -> Ubuntu VM Streamlit Dashboard
```

- Streamlit must be running inside the Ubuntu VM.
- VirtualBox NAT forwarding must forward Windows localhost port 8501 to the Ubuntu VM port 8501.
- Windows portproxy must expose the current Windows network IP on port 8501.
- The other system must be able to route to the Windows host IP.
- Previously captured SQLite data can still be displayed when live attacks are stopped, but the VM and dashboard must remain online.

6. Database Design

6.1 Live Dashboard Tables

Table	Main Fields	Retention
packets	timestamp, src, dst, protocol, src_port, dst_port, size, info, tcp_flags	Latest 5000 rows
alerts	timestamp, rule/msg, classification, priority, protocol, src/dst, snort_rule, iface, service, src_addr, src_port, dst_addr, dst_port, direction, pkt_len, action, gid, sid, rev, ttl, tos, tcp_flags, icmp_type, icmp_code, client/server packet and byte counters, raw_json	Latest 1000 rows

The Scapy packet collector inserts captured packet metadata directly into SQLite. Snort alerts are handled through the raw JSON alert log, which is parsed by the alert collector and inserted into the alerts table. This design separates general packet visibility from rule-based security alerts.

6.2 Rule Documentation Knowledge Database

The project includes a cleaned and normalized Snort rule documentation knowledge database. It is separate from the rolling-window packets and alerts tables and is used for rule lookup, research, and future LLM/RAG explanation work.

Table	Purpose	Rows / Relationship
rules	Main parent table with one row per Snort rule, including rule identity, header fields, direction_label, flow, service, metadata, and original rule_text.	4017 rows; primary key is gid + sid.
rule_documentation	Snort.org documentation fields such as category, explanation, what to look for, known usage, false positives, rule groups, and vulnerability text.	4017 rows; connected to rules by gid + sid.
rule_cves	Stores one CVE per rule instead of keeping comma-separated CVEs in one cell.	2278 rows; many CVEs can belong to one rule.
rule_mitre	Stores one MITRE ATT&CK mapping per rule with technique ID, tactic, and technique name.	207 rows; a rule can map to multiple MITRE techniques.
rule_references	Stores external references such as CVE, URL, Bugtraq, and Nessus references.	6735 rows; a rule can have many references.
rule_content_matches	Stores each Snort content option separately, including order and whether it is negated.	7413 rows; a rule can contain many content patterns.

7. Rule Dataset Preprocessing and Exports

The rule documentation dataset was preprocessed before being normalized. Empty strings, whitespace-only values, and placeholder text were converted to NULL. Scraper/debug fields such as http_status and fetched_at were removed from the cleaned dataset because they describe the scraping process rather than the rule itself. Noisy website footer text was also removed from documentation fields.

Preprocessing Step	Reason
Null normalization	Avoids confusing empty strings, whitespace, and placeholder values with real data.
Removal of scrape/debug fields	Fields such as http_status and fetched_at are useful for auditing but do not explain the rule.
Removal of noisy CVE additional information	The scraped field mostly contained website footer and loading text, so it was excluded from useful documentation context.
direction_label creation	Provides readable traffic-direction values such as source_to_destination and bidirectional.
CVE, MITRE, reference, and content normalization	Allows one rule to have many related CVEs, MITRE techniques, references, and content patterns without storing them as comma-separated cells.
JSON export keyed by SID	Allows the dashboard and future LLM/RAG systems to retrieve rule context quickly using the alert SID.

Folder	File	Purpose
data/raw/	rule_docs_review.xlsx	Review-stage rule documentation export.
data/preprocessed/	rule_docs_preprocessed.xlsx	Cleaned single-table preprocessed dataset.
data/normalized/	normalized_snort_rule_database.xlsx	Multi-sheet normalized rule database export.
data/json/	rule_docs_preprocessed_by_sid.json	JSON rule lookup dataset keyed by SID.

8. Feature Study: What the Fields Mean

General Feature	Meaning	Why It Matters
timestamp	When the packet or alert was recorded.	Builds timelines and supports incident sequence analysis.
msg / Alert Message	Human-readable Snort rule message.	Best field for quickly understanding what was detected.
classification	Broad category such as Misc activity or Attempted	Groups alerts by security meaning.

General Feature	Meaning	Why It Matters
	Information Leak.	
priority	Severity value; lower numbers are more serious.	Supports triage and dashboard severity metrics.
rule / gid / sid / rev	Snort rule identifier and revision.	Supports rule lookup, reproducibility, and signature explanation.
protocol / proto	Protocol such as TCP, UDP, or ICMP.	Determines which protocol-specific features should exist.
service	Application/service detected by Snort, such as HTTP, SSH, DNS, SNMP, or unknown.	Shows the type of service targeted or involved.
src_addr / dst_addr	Source and destination IP addresses.	Shows who sent the traffic and who received it.
direction	Alert traffic direction such as C2S or S2C.	Explains traffic flow during investigation.
pkt_len	Packet length that triggered the alert.	Provides low-level packet context.

8.1 Protocol-Specific Features

Protocol	Relevant Fields	Meaning
TCP	src_port, dst_port, tcp_flags, client/server packets and bytes	Explains connection attempts, SYN/FIN/NULL/Xmas patterns, service probing, and scan behavior.
UDP	src_port, dst_port, service	Useful for DNS/SNMP/DHCP-like probes and UDP scans. UDP has no TCP flags.
ICMP	icmp_type, icmp_code	Used for ping, echo reply, destination unreachable, and traceroute-like behavior. ICMP does not use ports.
HTTP	service=http, dst_port, msg/classification, packet info	Useful for web requests and suspicious HTTP rule matches.
DNS	protocol=DNS in Scapy, service=dns in Snort, dst_port=53	Useful for understanding DNS lookups and suspicious domain-related traffic.
ARP	src, dst, protocol=ARP, info	ARP has no IP ports, TTL, TCP flags, or ICMP type/code.

9. Signature Study: Snort Rules and Meanings

In Snort, signatures are rules. The project loads the local Snort Community Rules file, and live traffic is compared against those rules. Snort does not need to query an online signature database during packet processing because detections come from the rules already loaded locally.

Rule Field	Meaning
msg	Message printed when the rule matches. This becomes the main alert name on the dashboard.
gid	Generator ID. Identifies the Snort component that generated the event.
sid	Signature ID. Unique identifier for a Snort rule.
rev	Rule revision. This changes when the rule is updated.
classtype	Classification/category of the alert.
priority	Severity value assigned to the rule/event.
Header fields	Protocol, source/destination networks, ports, and traffic direction.
Rule options	Conditions such as content matches, ICMP type/code, TCP flags, service metadata, and references.

9.1 Example Signatures Observed

Signature	Meaning	Reason It May Trigger
PROTOCOL-ICMP Echo Reply	The victim replied to a ping request.	ICMP type 0 and code 0 match echo reply behavior. Usually low severity visibility.
PROTOCOL-ICMP Echo Request	A host is being pinged or probed.	ICMP type 8 indicates an echo request.
PROTOCOL-SNMP request tcp	Traffic probed SNMP-related service behavior.	Nmap or service probing touched SNMP-related TCP ports/services. Often indicates information gathering.
PROTOCOL-SNMP trap tcp	Traffic targeted SNMP trap-related behavior.	SNMP trap activity/probe is associated with management notification services.
PROTOCOL-SNMP AgentX/tcp request	Traffic targeted AgentX/SNMP agent communication.	Useful for identifying SNMP/management service probing.

10. Dashboard Components Implemented

Dashboard Panel	Status	Purpose
Version and page sidebar	Implemented in v3.0	Shows dashboard version and page navigation.
Metric cards	Implemented in v3.0	Highlights packet and alert numbers using compact cards.
Network Traffic Dashboard	Implemented in v3.0	Separates normal packet and host visibility from alert analysis.
Alert Dashboard	Implemented in v3.0	Focuses on Snort detections, alert details, and rule documentation.
Detailed Recent Snort Alerts	Implemented	Displays rich alert fields such as rule ID, classification, protocol, service, ports, direction, TTL, TCP flags, ICMP type/code, and flow counters.
Clickable alert-row rule lookup	Implemented in v3.0	User selects an alert row and the dashboard retrieves matching rule documentation from the JSON dataset using SID.
Rule Documentation from Preprocessed JSON	Implemented in v3.0	Displays clean rule documentation, category, explanation, CVEs, metadata, and rule text from the JSON dataset.
Snort JSON Feature Meaning	Implemented	Provides built-in meanings for alert JSON fields.
Inspect Selected Raw JSON Alert	Implemented	Displays the raw JSON alert for the selected alert.
Ruleset Status	Implemented	Shows Snort version, rules file path, enabled rule count, file timestamp, size, and SHA-256 hash.

11. Dashboard v3.0 Interface Changes

- Dashboard version is displayed in the interface as v3.0.
- The sidebar now uses page-style navigation instead of a dropdown.
- Normal traffic graphs are moved to the Network Traffic Dashboard page.
- Snort alert analysis is moved to the Alert Dashboard page.
- Alert service distribution was removed to reduce clutter because it mostly showed unknown service values.
- Service Distribution, Alert Protocol Distribution, Top Alert Source IPs, and Top Alert Destination Ports charts were made more compact.
- Metric values are shown in compact cards for clearer visibility.
- The old dropdown-based Signature / Rule Explanation flow was replaced with clickable/selectable alert-row interaction.
- Rule details now come from rule_docs_preprocessed_by_sid.json rather than forcing users to re-select an alert from a dropdown.
- The local Snort rule-file match section was removed from the user-facing explanation view to keep the focus on the curated preprocessed JSON rule dataset.

12. GitHub Repository and Versioning

The GitHub repository was reorganized to separate runtime source code, datasets, helper scripts, documentation, and screenshots. Dashboard versions are maintained using Git tags rather than duplicating dashboard folders.

Version	Description
dashboard-v1.0	First complete Streamlit IDS dashboard release.
dashboard-v2.0	Updated dashboard with Snort rule datasets, helper scripts, and documentation.
dashboard-v3.0	Page navigation, clickable alert-row JSON rule lookup, metric cards, compact charts, and cleaner rule documentation display.

```
git checkout dashboard-v1.0   # view previous release
git checkout main             # return to latest version
```

12.1 Repository Structure

```
Real-Time-Network-IDS-Dashboard/
├── README.md
├── requirements.txt
```

```
├── .gitignore
├── src/
│   ├── dashboard.py
│   ├── init_db.py
│   ├── live_packet_collector.py
│   └── live_alert_collector.py
├── data/
│   ├── README.md
│   ├── raw/
│   ├── preprocessed/
│   ├── normalized/
│   └── json/
├── scripts/
│   ├── README.md
│   ├── build_rule_docs_db.py
│   ├── repair_rule_docs_fetch.py
│   ├── create_rule_preprocessed_json.py
│   ├── export_preprocessed_rules.py
│   ├── export_normalized_database_excel.py
│   ├── create_rule_cves_table.py
│   ├── create_rule_mitre_table.py
│   ├── create_rule_references_table.py
│   ├── create_rule_content_matches_table.py
│   └── get_rule_context.py
├── docs/
└── screenshots/
```

13. Running the Project

13.1 Start Dashboard

```
cd /home/maheen/ids_streamlit_dashboard
streamlit run dashboard.py --server.address 0.0.0.0 --server.port 8501 --server.headless true
```

13.2 Start Packet Collector

```
cd /home/maheen/ids_streamlit_dashboard
sudo python3 live_packet_collector.py
```

13.3 Start Alert Collector

```
cd /home/maheen/ids_streamlit_dashboard
python3 live_alert_collector.py
```

13.4 Start Snort

```
mkdir -p /home/maheen/snort_logs

sudo snort -q \
-c /usr/local/etc/snort/snort.lua \
-R /usr/local/etc/snort/rules/snort3-community.rules \
-i enp0s8 \
-A alert_json \
--lua "HOME_NET = '192.168.56.20/32'; EXTERNAL_NET = '192.168.56.10/32'; alert_json = { fields = 'timestamp
iface proto service src_addr src_port dst_addr dst_port dir pkt_len action msg class priority rule gid sid
rev ttl tos tcp_flags icmp_type icmp_code client_pkts client_bytes server_pkts server_bytes' }" \
2>/tmp/snort_json_errors.txt | tee -a /home/maheen/snort_logs/alert_json.txt
```

14. Generating Test Traffic from Kali

Purpose	Command
Connectivity test	ping -c 4 192.168.56.20
ICMP traffic	ping -c 20 192.168.56.20
HTTP traffic	curl http://192.168.56.20
SSH connection check	nc -vz 192.168.56.20 22
TCP SYN scan	sudo nmap -sS -Pn -p 1-1000 192.168.56.20
Service detection scan	sudo nmap -sV -Pn 192.168.56.20
Aggressive scan	sudo nmap -A -Pn 192.168.56.20
UDP scan	sudo nmap -sU -Pn --top-ports 20 192.168.56.20

15. Snort Limitations and Management

Limitation	Why It Matters	Mitigation
Rule/signature dependency	Snort alerts only when traffic matches loaded rules or enabled inspector events.	Keep rules updated, validate with known lab traffic, and combine with Scapy all-traffic visibility.
Ruleset version dependency	Different rule revisions may detect, classify, or prioritize the same traffic differently.	Record Snort version, rules file timestamp, enabled rule count, and SHA-256 hash.
Community Rules coverage	Community Rules may not cover every test, exploit, or scan type.	Document verified alert types and avoid claiming complete detection coverage.
False positives / false negatives	Benign traffic may alert, and malicious traffic may be missed.	Tune rules, review context, and correlate with packet data.
Encrypted traffic	Snort cannot deeply inspect encrypted payloads unless decrypted before inspection.	Use metadata, endpoint logs, DNS/proxy logs, or TLS inspection where appropriate.
Performance	Large traffic volume and many rules can cause drops or delays.	Monitor drops, tune rules, scale sensors, and avoid unnecessary full-payload logging.

16. Project-Specific Limitations

Project Limitation	Impact	Future Improvement
Collectors must run	Alerts in alert_json.txt do not appear in the dashboard until live_alert_collector.py inserts them into SQLite.	Add a collector/database health panel or systemd services.
SQLite backend	Good for the lab, but not ideal for enterprise storage or many sensors.	Use PostgreSQL, Elastic, or SIEM integration.
Three-VM virtual lab	Simplified compared with real networks.	Test with more hosts, services, and background traffic.
Limited alert-packet correlation	Alerts and packets are stored separately.	Correlate by timestamp, protocol, IPs, ports, and direction.
Network segmentation	Some wired/wireless networks may not reach the Windows host IP.	Use the same routable network, lab server, or approved VPN/overlay network.

17. Ruleset Maintenance Plan

- Record the Snort engine version using snort -V.
- Record the Community Rules file path, file size, last modified date, and enabled rule count.
- Store a SHA-256 hash of snort3-community.rules for reproducibility.
- Maintain a changelog whenever rules are updated.
- After each ruleset update, replay or regenerate known lab traffic to verify expected alerts still appear.
- Use the dashboard ruleset status panel to confirm which local ruleset produced the alerts.

18. Future Enhancements

- Add a collector/database health panel showing collector status, alert_json.txt status, and database counts.
- Add alert-to-packet correlation using timestamp, protocol, IPs, ports, and direction.
- Add exportable reports for alerts and packet summaries.
- Add email, Slack, or dashboard notifications for priority 1 and priority 2 alerts.
- Add optional anomaly detection to complement signature-based Snort alerts.
- Add authentication for shared dashboard access.
- Add support for multiple sensors or larger databases such as PostgreSQL or Elastic.
- Deploy on a dedicated lab-accessible server for more stable access.
- Integrate the JSON/normalized rule dataset with an LLM/RAG explanation module.

19. Conclusion

The current implementation is more advanced than the original version. It now uses Snort JSON alert output, stores richer alert features, keeps rolling-window SQLite storage, and includes a curated Snort rule documentation dataset. Scapy provides general packet visibility, while Snort provides rule-based alerting.

Dashboard v3.0 improves usability by separating the interface into Alert Dashboard and Network Traffic Dashboard pages, adding page-style sidebar navigation, introducing metric cards, making charts more compact, and replacing dropdown-based rule explanation with clickable alert-row selection. Rule details are retrieved from the preprocessed JSON dataset, making the alert explanation workflow easier for users and analysts.

The project is now organized for GitHub with version tags, dataset folders, helper scripts, documentation, and future LLM/RAG integration support.